Hierarchical Parameter Estimation

Mathematical Formalization of the Custom Metropolis-within-Gibbs MCMC

Antigravity AI Pipeline

August 2026

Abstract

This document formally specifies the custom Hierarchical Markov Chain Monte Carlo (MCMC) sampler implemented in C++ to estimate the parameters of the Unified Cortico-Cerebellar Model (ECCM). We mathematically detail the structural divergence between this sampler and the default *rstan* No-U-Turn Sampler (NUTS), explaining why standard Probabilistic Programming Languages (PPLs) fail on deep recurrent manifolds.

1 The Failure of Stan and NUTS on Recurrent Topologies

Stan utilizes Hamiltonian Monte Carlo (HMC) guided by the No-U-Turn Sampler (NUTS). This requires the algorithmic extraction of the exact gradient of the log-posterior with respect to the parameters via Automatic Differentiation (AutoDiff).

Intuition: Stan acts like a ball rolling down a multi-dimensional hill; to know which way to roll, it must calculate the exact slope (gradient) of the hill at every point.

Mathematical Impossibility: The ECCM possesses a 1024-node Granule layer and a 160-node continuous memory register. Calculating the likelihood requires iterating forward over $T \approx 1000$ trials. For Stan to calculate the gradient, it must unroll the recurrent state network over all 1000 trials and backpropagate the error through $1000 \times 1024 \times 160$ connections. This "Backpropagation Through Time" (BPTT) shatters the AutoDiff tape memory, causing instantaneous out-of-memory (OOM) failures or infinite compilation times. Stan cannot natively handle arbitrarily deep discrete-time recurrent manifolds.

2 The Custom Metropolis-within-Gibbs MCMC

To bypass gradient computation, we implemented a custom **Metropolis-within-Gibbs** sampler natively in C++.

Intuition: Instead of calculating the slope (gradient), this sampler blindly takes a random step in the dark (Random Walk). If the new location is higher (better likelihood), it steps there. If it's lower, it rolls a weighted die to decide whether to step anyway, preventing it from getting stuck in local traps.

2.1 Hierarchical Priors

The model utilizes a two-level hierarchical geometry. Let $\theta_{s,p}$ be the parameter p for subject s. To operate in an unbounded Euclidean space, bounded parameters (like $\eta \in (0, 1)$ or $a > 0$) are

transformed using logits and logarithms.

$$\text{Hyper-Mean:} \quad \mu_p \sim \mathcal{N}(0, 10^2) \tag{1}$$

$$\text{Hyper-Variance:} \quad \sigma_p \sim \text{HalfCauchy}(0, 5) \tag{2}$$

$$\text{Subject Level:} \quad \theta_{s,p} \sim \mathcal{N}(\mu_p, \sigma_p^2) \tag{3}$$

2.2 The Metropolis Block (Subject-Level)

For a given subject s and parameter p, we propose a new value via a Gaussian random walk:

$$\theta_{s,p}^* \sim \mathcal{N}(\theta_{s,p}^{(current)}, \sigma_{prop}^2) \tag{4}$$

The acceptance ratio α evaluates the Curvature-Penalized Likelihood $\mathcal{L}_{penalized}$ (the behavioral fit) combined with the hierarchical prior penalty:

$$\alpha = \min\left(1, \frac{\exp(-\mathcal{L}_{pen}^*) \cdot P(\theta_{s,p}^* \mid \mu_p, \sigma_p)}{\exp(-\mathcal{L}_{pen}^{(current)}) \cdot P(\theta_{s,p}^{(current)} \mid \mu_p, \sigma_p)}\right) \tag{5}$$

2.3 The Gibbs/Metropolis Block (Group-Level)

After sampling all subject parameters, the group-level hyperparameters μ_p and σ_p are updated by evaluating their likelihood against the set of all current subject parameters $\boldsymbol{\theta}_p = \{\theta_{1,p}, \ldots, \theta_{N,p}\}$:

$$P(\mu_p^* \mid \boldsymbol{\theta}_p, \sigma_p) \propto \left(\prod_{s=1}^{N} \mathcal{N}(\theta_{s,p} \mid \mu_p^*, \sigma_p^2)\right) \cdot \mathcal{N}(\mu_p^* \mid 0, 10^2) \tag{6}$$

By separating the Subject update from the Group update, the Metropolis-within-Gibbs structure dramatically reduces the dimensionality of the proposal space, allowing the random walk to converge efficiently on a 72-dimensional surface without requiring gradient calculus.